SISTEM DE MANAGEMENT CERTIFICAT
ID C131781
ISO 9001 ISO 14001

Liceul Teoretic Jean Monnet

AskDuino

Proiect în vederea obţinerii atestatului profesional la informatică

Profesor coordonator

PAUL ENACHE

Candidat

Stroescu Eliana

Clasa a XII-a C

Bucuresti

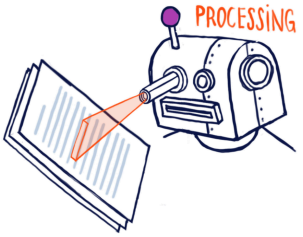
Mai 2026

Cuprins:

1. Introducere **2**
1.1. Prezentarea generala a domeniului ales 2
1.2. Justificarea alegerii temei & scopul proiectului 4
1.3. Descrierea tehnologiilor folosite 5
2. Prezentarea proiectului **6**
2.1. Titlul proiectului 6
2.2. Mediul de dezvoltare 6
2.3. Cerințe tehnice 6
2.4. Kit de instalare 7
2.5. Cum funcționează? 7
2.6. Manual de utilizare 8
2.6.1. Pornirea sistemului 8
2.6.2. Utilizarea aplicației 9
3. Documentație tehnică **9**
3.1. Arhitectură & Structură 9
3.2. Fluxul aplicatiei 9
3.3. Elemente relevante din cod 10
3.3.1. MAIN 10
1. Functia app_main 10
2. Functia gestioneaza_chat_post 11
3.3.2. CHAT 14
1. Structura chat_entry_t 14
2. Constanta ETICHETA_CHAT & PROMPT_SISTEM 14
3. Funcția adauga_istoric_chat 14
4. Funcția istoric_chat_goleste 15
5. Funcția chat_client_apel 16
3.3.3. LED 20
1. Funcția aplica_culoare_led 20
2. Funcția parseaza_culoare_hex 21
3. Funcția parseaza_culoare_nume 23
4. Funcția led_ctrl_proceseaza_comanda 24
3.3.4. UI 24
1. Interfața grafică a aplicației 24
2. Funcția append (role, text) 25
3.3.5. C3 OLED 25
1. Functia display_show_icon 25
2. Functia c3_client_proceseaza_comanda 26
4. Concluzia **28**
5. Bibliografie **28**

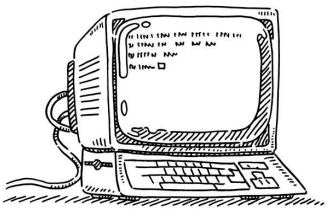
1. Introducere

1.1. Prezentarea generala a domeniului ales

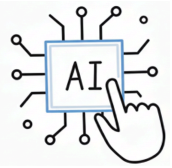


În ultimele decenii, tehnologia informației a trecut de la statutul de disciplină tehnică specializată la unul de infrastructură globală, indispensabilă funcționării societății contemporane. Evoluția calculatoarelor, a internetului și a aplicațiilor software a redefinit modul în care oamenii comunică, învață, lucrează și se raportează la lumea din jur. Dacă primele calculatoare din anii ’50 și ’60 necesitau încăperi dedicate și echipe întregi de specialiști pentru operare, în prezent computerele sunt integrate în smartphone-uri, ceasuri inteligente, automobile, electrocasnice și chiar în jucării. Această miniaturizare și democratizare a accesului la tehnologie au creat premisele unei societăți profund digitalizate.

Internetul a avut un rol decisiv în această transformare. De la o rețea experimentală destinată universităților și laboratoarelor, a devenit platforma globală prin care circulă informația, comerțul, serviciile financiare, comunicarea interpersonală și divertismentul. Rețelele sociale au modificat dinamica socială, permițând formarea comunităților virtuale și transmiterea instantanee a ideilor, dar în același timp au ridicat probleme legate de manipulare informațională, filtrarea opiniilor și expunerea excesivă la conținut.



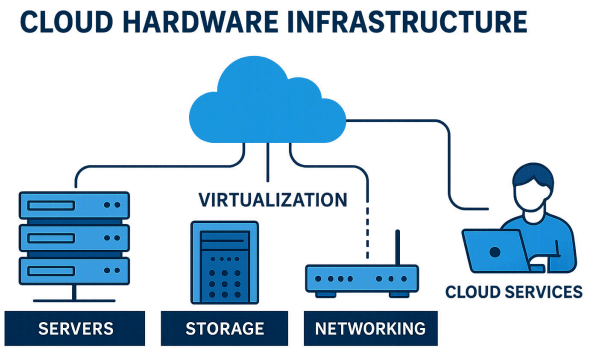
În paralel, apariția calculatoarelor personale în anii ’80 și, ulterior, a smartphone-urilor după 2007, a schimbat paradigma interacțiunii oamenilor cu mașinile. Relația a devenit mobilă, personalizată și permanentă. Utilizatorii nu mai accesează ocazional calculatoarele; ei trăiesc într-un flux continuu de informații, notificări și servicii. Acest mediu digital este completat de software inteligent, bazat tot mai mult pe tehnici de inteligență artificială (IA).



IA a progresat accelerat datorită creșterii puterii de calcul, a disponibilității masive de date și a algoritmilor perfecționați. Modelele moderne pot interpreta imagini, text, sunet și secvențe video, pot rezolva sarcini complexe și pot genera conținut original. Modelele mari de limbaj (LLM) reprezintă o etapă importantă în această evoluție, facilitând interacțiunea naturală între oameni și sisteme informatice. Asistenții conversaționali, instrumentele de programare asistată sau sistemele de traducere automată nu mai sunt doar demonstrații tehnice, ci elemente adoptate în producție, cu impact asupra industriei și educației.

Transformarea digitală depășește simpla automatizare. Ea produce mutații economice și culturale. Serviciile de streaming au transformat industria media, comerțul electronic a schimbat retailul, telemedicina reformează accesul la sănătate, iar fintech-urile redefinesc banking-ul. Totodată, pe măsură ce datele devin resursa esențială a economiei, securitatea cibernetică capătă importanță strategică. Amenințările digitale, de la atacuri ransomware până

la furt de identitate sau manipulare informațională, obligă la dezvoltarea unor strategii globale de protecție, legislație adecvată și educație tehnică.



De asemenea, tehnologiile cloud sunt esențiale în dezvoltarea aplicațiilor moderne, oferind posibilitatea de a stoca și procesa date la distanță, fără a depinde de resursele hardware locale. Cloud-ul permite rularea aplicațiilor scalabile, accesibile de oriunde, existând o conexiune la internet, fiind utilizat pe scară largă în domeniul educației, al afacerilor și al cercetărilor. De asemenea, mediile cloud permit testarea și dezvoltarea rapidă a aplicațiilor, reducând timpul necesar lansării pe piață.

Totuși, tehnologia nu este doar o sursă de beneficii; ea ridică și dileme fundamentale. Inteligența artificială poate amplifica biasuri culturale sau statistice dacă este antrenată pe date neechilibrate. Automatizarea ridică întrebări despre viitorul locurilor de muncă, despre reconversia profesională și despre echitatea distribuției bunăstării generate de tehnologie. Deciziile algoritmice profesionale sau administrative cer transparență, auditabilitate și responsabilitate.

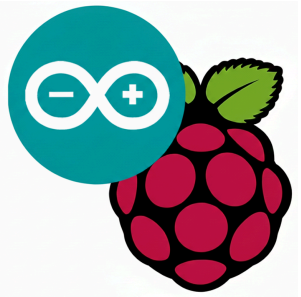
Un alt fenomen major este apariția dispozitivelor conectate (Internet of Things – IoT), alimentate de rețele de comunicații de mare viteză precum 5G și, în viitor, 6G. Casele inteligente, infrastructură urbană conectată și vehiculele autonome demonstrează cum datele colectate de senzori pot fi procesate în timp real pentru a optimiza traficul, energia, resursele și confortul. Acest sistem socio-tehnic creează valori noi, dar cere și standarde comune, interoperabilitate și atenție la confidențialitatea utilizatorilor.



Miniaturizarea accelerată a componentelor electronice a deschis și o dimensiune educațională și experimentală. Platformele precum Arduino, Raspberry Pi, ESP32 sau micro:bit permit construirea rapidă de prototipuri hardware, accesibile elevilor, studenților și antreprenorilor. Chiar dacă resursele hardware sunt relativ limitate, posibilitatea de a controla senzori, motoare și actuatoare a democratizat inovația tehnică, generând noi valuri de creativitate în domeniul embedded. Integrarea cu servicii cloud și IA permite realizarea de proiecte care acum două decenii ar fi necesitat laboratoare industriale.

Educația și cultura sunt profund remodelate de această transformare digitală. Platformele de învățare online, bibliotecile digitale, laboratoarele virtuale și resursele open-source extind accesul la cunoaștere. Alfabetizarea digitală devine o competență fundamentală, comparabilă cu alfabetizarea tradițională. Tinerii cresc într-un mediu tehnologic care favorizează învățarea multimodală, experimentarea rapidă și accesul instant la informații. Creativitatea se exprimă

prin instrumente de design digital, producție audio-video, programare și IA generativă, care reduc barierele tehnice și încurajează expresia artistică.



În concluzie, IT-ul, calculatoarele și inteligența artificială nu sunt doar mijloace de productivitate, ci componente structurale ale societății. Provocarea este să se mențină un echilibru între inovație, etică, protecția datelor și incluziune socială, astfel încât tehnologia să rămână orientată spre binele comun.

1.2. Justificarea alegerii temei & scopul proiectului

Motivația realizării acestui proiect pornește din posibilitatea de a dezvolta un echipament cu resurse puține și de a răspunde unor probleme actuale într-un mod în care să fie de folos (explorarea educațională care mă poate ajuta să învăț, nu neapărat să fie util), pentru a aplica cunoștințele dobândite la disciplina informatică într-un mod practic utilizând un dispozitiv hardware cu resurse puține, dar suficiente pentru ceea ce avem nevoie.

Proiectul îmbină programarea, utilizarea unei rețele locale, a internetului și a inteligenței artificiale cu controlul unui sistem Arduino, demonstrând modul în care conceptele actuale din domeniul informaticii pot fi implementate la nivel educațional.

Pe de o parte, tehnologia aduce avantaje incontestabile: creșterea productivității, accesul la educație și sănătate, optimizarea resurselor, reducerea risipei, posibilitatea colaborării globale și democratizarea cunoașterii. Pe de altă parte, ea generează probleme precum supravegherea excesivă, manipularea informațională, dependența digitală, vulnerabilitatea la atacuri cibernetice și apariția inegalităților generate de accesul diferențiat la tehnologie.

Alegerea temei este determinată și de importanța alfabetizării digitale. Dacă în secolul XX, alfabetizarea presupunea capacitatea de a citi și scrie, secolul XXI include și capacitatea de a naviga mediile digitale, de a înțelege algoritmi și platforme, de a evalua critic sursele de informații și de a utiliza instrumente inteligente în mod responsabil. Această competență devine esențială pe măsură ce tehnologia penetrează aproape toate domeniile vieții.

Deci, tema este relevantă deoarece tehnologia nu se mai limitează la calculatoare clasice. Ea include sisteme embedded, rețele IoT, dispozitive autonome, AI generativă, cloud computing și interacțiunea complexă dintre hardware și software.

În această ecuație, internetul reprezintă infrastructura de bază. Este una dintre cele mai importante invenții ale omului, transformând modul în care comunicăm, lucrăm, învățăm și ne organizăm societățile. Dezvoltarea AI ar fi fost imposibilă fără internet și cloud, deoarece modelele moderne necesită volume enorme de date și resurse de calcul distribuite.

1.3. Descrierea tehnologiilor folosite

C++ este un limbaj de programare creat pentru a oferi atât performanță ridicată, cât și flexibilitate în dezvoltarea aplicațiilor complexe. Apărut ca o extensie a limbajului C, acesta a fost conceput pentru a păstra viteza și eficiența acestuia, adăugând în același timp concepte moderne de programare, precum programarea orientată pe obiect (OOP).



Unul dintre cele mai importante avantaje ale limbajului este controlul detaliat asupra resurselor hardware, de a gestiona memoria în mod explicit, fiind folosit frecvent în dezvoltarea sistemelor de operare, a aplicațiilor embedded, a motoarelor de jocuri și a software-ului de timp real. C++ admite mai multe paradigme de programare, printre care programarea procedurală, orientată pe obiecte și generică. Prin utilizarea claselor, obiectelor și șabloanelor (templates), codul poate fi organizat eficient, ușor de extins și utilizat, contribuind la realizarea unor aplicații robuste și scalabile.

De-a lungul timpului, limbajul C++ a evoluat constant prin standarde succesive, care au introdus funcționalități moderne și au îmbunătățit siguranța și productivitatea dezvoltatorilor. Astfel, C++ rămâne un limbaj actual, utilizat atât în industrie, cât și în mediul academic, apreciat pentru eficiența, puterea și versatilitatea sa. Deși poate fi dificil pentru începători, stăpânirea lui oferă o bază solidă pentru înțelegerea modului în care funcționează software-ul la nivel profund și pentru dezvoltarea unor aplicații performante.

HTML (HyperText Markup Language) reprezintă fundamentul oricărei pagini web, prin care se structurează conținutul unei pagini, definind elemente precum titlurile, paragrafele, imaginile, linkurile sau formularele. Fără HTML, informația afișată în browser nu ar putea fi organizată și interpretată corect.

Importanța HTML-ului constă în oferirea bazei pe care se construiesc toate celelalte tehnologii web, precum CSS și JavaScript. Dacă CSS se ocupă de aspectul vizual, iar JavaScript de interactivitate, HTML stabilește structura și sensul conținutului. De asemenea, un cod HTML contribuie la accesibilitatea site-urilor și la indexarea corectă în motoarele de căutare, fiind un element indispensabil al web-ului, fiind punctul de plecare pentru orice aplicație sau site online.



2. Prezentarea proiectului

2.1. Titlul proiectului: AskDuino

Denumirea proiectului provine din combinarea cuvântului „Ask”, termen provenit din limba engleză, cu sensul de „a întreba”, și placa de dezvoltare, „Arduino”. Numele reflectă funcționalitatea principală a proiectului, aceea de a permite utilizatorului să adreseze întrebări sau comenzi dispozitivului, folosind un sistem de comunicare.

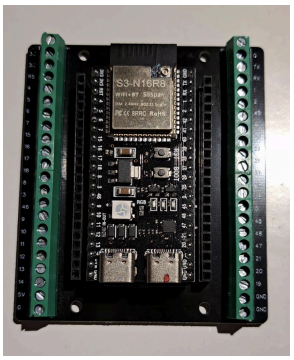
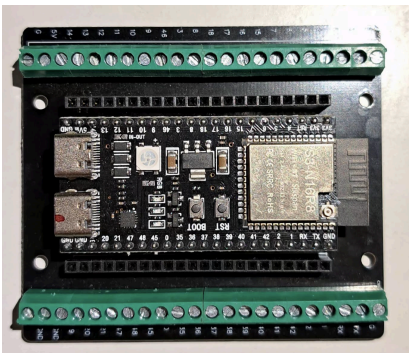
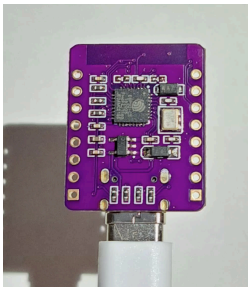
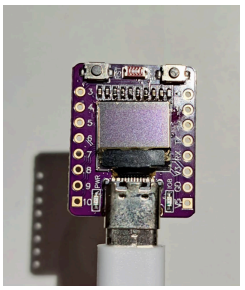
2.2. Mediul de dezvoltare

★ **Mediu de dezvoltare:** Visual Studio Code
★ **Limbaj de programare:** C++
★ **Platformă hardware:** ESP32 (placa fizică)

2.3. Cerințe tehnice (hardware)

Pentru asigurarea funcționării corecte a proiectului, sunt necesare:

- ❖ Placă de dezvoltare ESP32-S3 cu kit adaptor de extensie, modul MCU ESP32-S3-WROOM-1 N16R8, Wi-Fi 2.4G și Bluetooth, 44 pini, interfață USB Type-C, cu LED-RGB
- ❖ Placă de dezvoltare ESP32-C3 cu ecran OLED de 0,42 inch, antenă ceramică, placă de dezvoltare ESP32 SuperMini cu Wi-Fi și Bluetooth
- ❖ Două cabluri USB Type-C pentru conectare

2.4. Kit de instalare (cerințe software)

★ Necesitați:
  - Acces la Wi-Fi
  - O cheie validă OpenAI
★ Se instalează VSCode
★ Se instalează ESP-IDF versiunea 5.1.6.
★ Se rulează comanda **.\esp-idf\export.bat**
★ Se rulează comanda **idf.py set-target esp32**
★ Cu comanda **idf.py menuconfig** se deschide meniul de configuratie și se adaugă:
  - App config -> Wi-Fi SSID/PASS (nume și parola wifi)

- App config -> OpenAI API Key / Model (cheie și model OpenAI)
- App config -> WS2812 LED GPIO (pinul în caz ca se folosește un led extern)

★ Se ruleaza comanda **idf.py build**
★ În *device manager* se caută portul COM la care este conectată plăcuța
★ Se rulează comanda **idf.py -p <PORT> flash monitor** unde PORT este înlocuit de portul COM aflat la pasul anterior
★ După boot apare mesajul: **Chat ready: http://<IP>/**

2.5. Cum funcționează?

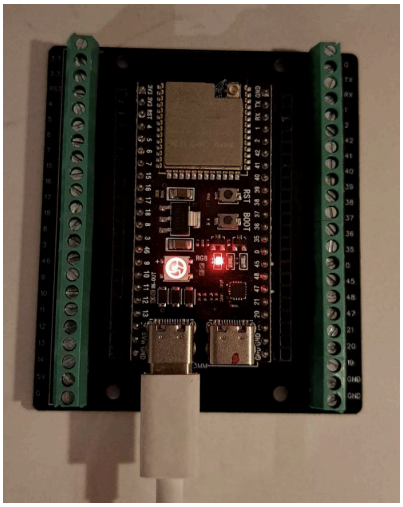
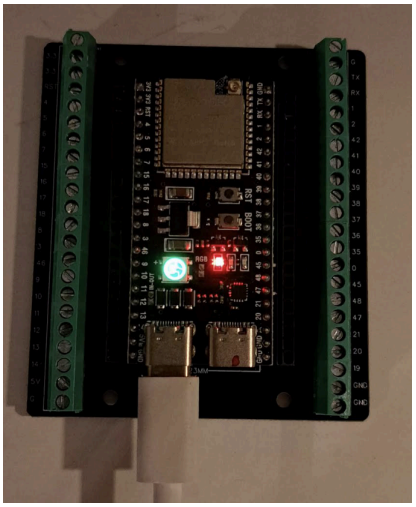
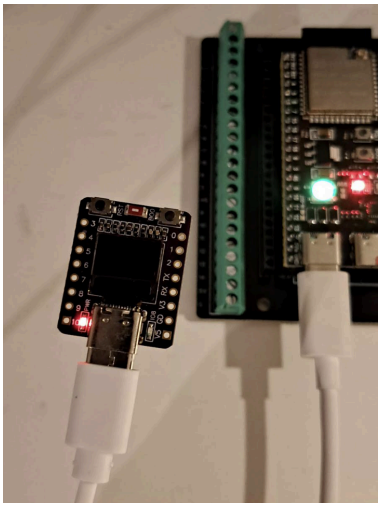
Acest proiect transformă un ESP32-S3 într-un mini laborator interactiv:

1. Placa se conectează la Wi-Fi, găzduiește o interfață web modernă și expune un endpoint WebSocket pentru chat.
2. Mesajele sunt trimise la OpenAI (model configurabil), iar răspunsurile sunt afișate în timp real.
3. Poți controla LED-ul RGB (WS2812 / NeoPixel) direct prin comenzi în română: „aprinde ledul roșu”, „led #00ffaa”, „stinge ledul” și ecranul OLED pentru a afișa emoticoane sau forme geometrice, spre exemplu: „vesel”, „trist”, „patrat”, „ triunghi”, „cerc”, „ stea”, etc.

2.6. Manual de utilizare

2.6.1. Pornirea sistemului

1. Se conectează placa ESP32 prin cablu USB la o sursă de energie (ex: calculator, baterie). Un port USB standard (PC, laptop) sau un adaptor USB de 5 V/1 A (5 W) este suficient.
2. Se va conecta la o adresă de WI-FI deja configurată cu numele și parola.
3. Dacă nu este detectată, intră în mod AP (Access Point) și pornește un portal web de configurare Wi-Fi, care permite setarea Wi-Fi-ului direct din browser.
4. După schimbarea LED-ului din roșu în verde, ce confirmă conectarea dispozitivului, este gata de folosit.

5. Se va deschide o pagină nouă de browser, iar în bara de căutare se va scrie IP-ul dispozitivului.
6. După încărcare, va apărea chat-ul cu statusul „Se conectează…”; după ce se stabilizează website-ul și statului se schimbă în „*Conectat”*, conversația este disponibilă.



2.6.2. Utilizarea aplicației (scrierea în chat)

★ După conectarea la pagina web, utilizatorul are posibilitatea de a scrie mesaje în limba română.
★ Există un prompt cu setări disponibile (exemplificarile făcute sunt la capitolul 2.5.3.), pentru:
  - schimbarea stării LED-ului
  - afișarea unei forme geometrice
  - afișarea unui emoticon
★ Butonul *Reset* șterge istoricul și resetează promptul.

LED-ul este controlat local dacă mesajul conține cuvântul-cheie „*led”/„rgb”/„lumină”* și o valoare de culoare validă (numele unei culori suportate sau un cod hex). Dacă nu există potrivire, mesajul este trimis la OpenAI.

3. Documentație tehnică

3.1. Arhitectură & Structură

❖ *main/main.cpp* – inițializează Wi-Fi-ul, serverul HTTP/WS și orchestrează schimbul dintre UI, ChatGPT și LED.
❖ *main/chat_client.cpp* – construiește payload-ul pentru OpenAI, gestionează istoricul conversației și parsează răspunsurile.

- *main/led_ctrl.cpp* – se ocupă de inițializarea LED-ului (WS2812 + fallback GPIO) și de comenzile locale.*main/ui_html.cpp* – conține interfața web (HTML/CSS/JS) în format C string.
- *main/idf_component.yml* – declară dependențele managed (ex: *espressif/ led_strip*).
- *sdkconfig.defaults* – valori implicite pentru configurație (flash 16MB, HTTPD, bundle certificate).
- *source_esp_idf.sh* – ajutator pentru încărcarea mediului ESP-IDF fără a ieși din repo.
- *display.c* – gestionează inițializarea și controlul ecranului OLED, oferind funcții pentru afișarea de iconițe (emoticoane și forme geometrice); conține mecanisme de temporizare pentru ștergerea automată după inactivitate.

3.2. Fluxul aplicatiei

- **Boot + LED de stare** – LED-ul pornește roșu și își schimbă culoarea în funcție de starea aplicației (albastru când Wi-Fi este online, verde când serverul web rulează).
- **Conectare la Wi-Fi** – *initializeaza_wifi()* citește SSID/PASS din menuconfig și așteaptă bitul *WIFI_CONNECTED_BIT* înainte de a continua.
- **Server HTTP/WS** – UI-ul din *ui_html.cpp* este livrat pe „/”, iar WebSocket-ul „/ws” gestionează chat-ul live; „/chat” oferă aceeași funcționalitate via REST.
- **Proces mesaje**–fiecare mesaj trece prin *led_ctrl_proceseaza_comanda()*; dacă nu este o comandă LED, *chat_client_apel()* construiește payload-ul OpenAI și așteaptă răspunsul.
- **Istoric scurt** – ultimele mesaje (max 8) sunt păstrate pentru a oferi context conversației fără a consuma excesiv RAM.

3.3. Elemente relevante din cod

3.3.1. MAIN

1. **Functia *app_main***

Funcția *app_main* este punctul de pornire al aplicației pe ESP32 și coordonează întregul proces de inițializare și rulare a sistemului. La început, funcția inițializează memoria non-volatilă (NVS), care este folosită pentru a stoca configurația Wi-Fi.

```
esp_err_t cod_eroare = nvs_flash_init();
if (cod_eroare == ESP_ERR_NVS_NO_FREE_PAGES || cod_eroare == ESP_ERR_NVS_NEW_VERSION_FOUND) {
    ESP_ERROR_CHECK(nvs_flash_erase());
    ESP_ERROR_CHECK(nvs_flash_init());
}
```

Imediat după aceea, este inițializat LED-ul de stare prin apelul *led_ctrl_initializeaza*. LED-ul este aprins în roșu, semnalizând faza de boot și faptul că sistemul se află în proces de inițializare.

```
// Inițializăm LED-ul de stare cât mai devreme pentru feedback vizual.
ESP_ERROR_CHECK(led_ctrl_initializeaza());
led_ctrl_seteaza_culoare(10, 0, 0); // roșu pe durata boot-ului
```

Urmează inițializarea clientului BLE (Bluetooth Low Energy) în *(c3_client_init)*. Această etapă nu este critică pentru funcționarea aplicației, astfel că, în cazul unui eșec, sistemul continuă să ruleze fără acest modul, afișând doar un mesaj de avertizare în jurnal.

```
// Pornim clientul BLE către C3 (nu blocăm dacă eșuează).
esp_err_t err_c3 = c3_client_init();
if (err_c3 != ESP_OK) {
    ESP_LOGW(ETICHETA_JURNAL, "C3 BLE init failed (%s) — continui fără el", esp_err_to_name(err_c3));
}
```

Apoi, funcția încearcă să conecteze dispozitivul la o rețea Wi-Fi, apelând la *initializeaza_wifi*. Dacă există rețele configurate și una dintre ele este disponibilă, conexiunea se stabilește, iar LED-ul este schimbat în albastru, indicând că dispozitivul este online.

```
ESP_LOGI(ETICHETA_JURNAL, "Connecting to Wi-Fi...");
if (initializeaza_wifi() != ESP_OK) {
    ESP_LOGW(ETICHETA_JURNAL, "Nu s-a conectat la retelele preferate; intru in portal de configurare.");
    led_ctrl_seteaza_culoare(20, 5, 0); // portocaliu in modul AP
    if (porneste_portal_config() != ESP_OK) {
        led_ctrl_seteaza_culoare(20, 0, 0);
        return;
    }
    while (true) {
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
led_ctrl_seteaza_culoare(0, 0, 20); // albastru când conexiunea e activă
```

Odată ce conexiunea Wi-Fi este stabilită cu succes, funcția pornește serverul web principal prin *porneste_server_web*. Dacă serverul web pornește fără erori, LED-ul este schimbat în verde, semnalizând că aplicația este complet funcțională și gata de utilizare.

```
// Serverul HTTP deservește UI-ul, endpoint-ul REST și upgrade-ul WebSocket.
ESP_LOGI(ETICHETA_JURNAL, "Starting web server...");
if (porneste_server_web() != ESP_OK) {
    ESP_LOGE(ETICHETA_JURNAL, "Failed to start HTTP server");
    led_ctrl_seteaza_culoare(20, 0, 0);
    return;
}

led_ctrl_seteaza_culoare(0, 20, 0); // verde când serverul e gata
```

Funcția va obține mai apoi adresa IP a dispozitivului și o va afișa în jurnal, pentru a putea fi accesată din browser. După această etapă, *app_main* intră într-o buclă infinită cu vTaskDelay, menținând task-ul principal activ, în timp ce restul logicii aplicației rulează asincron în task-urile FreeRTOS (Wi-Fi, server HTTP, WebSocket, apeluri OpenAI).

```
char ip_text[16] = "0.0.0.0";
if (s_interfata_sta) {
    esp_netif_ip_info_t informatii_ip;
    if (esp_netif_get_ip_info(s_interfata_sta, &informatii_ip) == ESP_OK) {
        esp_ip4addr_ntoa(&informatii_ip.ip, ip_text, sizeof(ip_text));
    }
}

ESP_LOGI(ETICHETA_JURNAL, "Chat ready: http://%s/ (model: %s)", ip_text, CONFIG_APP_OPENAI_MODEL);

// Task-ul principal nu mai are treabă; celelalte task-uri FreeRTOS se ocupă de rețea.
while (true) {
    vTaskDelay(pdMS_TO_TICKS(1000));
}
```

2. Functia *gestioneaza_chat_post*

Funcția *gestioneaza_chat_post* este responsabilă de procesarea mesajelor trimise prin endpoint-ul REST POST *„/chat”* și reprezintă varianta de rezervă pentru comunicarea cu browserul atunci când WebSocket-ul nu este folosit. Practic, ea face legătura dintre interfața web și logica aplicației, ocupându-se atât de validarea cererilor, cât și de obținerea răspunsului corect.

La început, funcția verifică dacă cererea primită are un corp valid și dacă dimensiunea acestuia se încadrează în limitele stabilite.

```
static esp_err_t gestioneaza_chat_post(httpd_req_t *cerere)
{
    if (cerere->content_len <= 0 || cerere->content_len > MAX_CORP_CERERE) {
        return trimite_json_eroare(cerere, "400 Bad Request", "Cererea este prea mare.");
    }
```

După validare, funcția citește întregul corpul cererii HTTP. Citirea se face controlat, în mai multe etape, dacă este necesar, până când toți octeții sunt primiți. Odată ce datele au fost citite complet, ele sunt interpretate ca text și parsate într-un obiect JSON. Dacă acest pas eșuează, funcția consideră cererea invalidă și răspunde cu o eroare clară pentru client.

```
    int octeti_primiti = 0;
    while (octeti_primiti < cerere->content_len) {
        int rezultat_primire = httpd_req_recv(cerere,
                                              buffer_cerere + octeti_primiti,
                                              cerere->content_len - octeti_primiti);
        if (rezultat_primire <= 0) {
            if (rezultat_primire == HTTPD_SOCK_ERR_TIMEOUT) {
                continue;
            }
            free(buffer_cerere);
            return ESP_FAIL;
        }
```

Odată obținut JSON-ul, funcția verifică mai întâi dacă există o acțiune specială, cum ar fi reset. În acest caz, istoricul conversației este șters, iar funcția trimite un răspuns simplu de confirmare.

```
cJSON *actiune = cJSON_GetObjectItem(obiect_json, "action");
if (cJSON_IsString(actiune) && strcmp(actiune->valuestring, "reset") == 0) {
    istoric_chat_goleste();
    cJSON_Delete(obiect_json);
    return trimite_json_kv(cerere, "status", "reset");
}
```

Dacă nu este vorba despre o acțiune de reset, funcția caută câmpul message, care reprezintă textul introdus de utilizator. Mesajul este verificat pentru a exista, a nu fi gol și a nu depăși lungimea maximă permisă. După validare, mesajul este copiat într-un buffer local, pentru a fi procesat mai departe în siguranță.

```
cJSON *mesaj = cJSON_GetObjectItem(obiect_json, "message");
if (!cJSON_IsString(mesaj) || strlen(mesaj->valuestring) == 0) {
    cJSON_Delete(obiect_json);
    return trimite_json_eroare(cerere, "400 Bad Request", "Mesaj lipsă.");
}
```

În continuare, funcția încearcă să rezolve mesajul local, fără a apela la OpenAI. Mai întâi sunt verificate comenzile destinate altor module (de exemplu, comenzi pentru LED). Dacă una dintre aceste funcții recunoaște și procesează mesajul, răspunsul este generat local, istoricul conversației este actualizat, iar clientului i se trimite imediat un răspuns JSON. Această abordare face aplicația mai rapidă și reduce dependența de serviciile externe.

```
char *raspuns_local = NULL;
bool gestionat_local = c3_client_proceseaza_comanda(copie_utilizator, &raspuns_local);
if (!gestionat_local) {
    gestionat_local = led_ctrl_proceseaza_comanda(copie_utilizator, &raspuns_local);
}
if (gestionat_local) {
    cJSON_Delete(obiect_json);
    adauga_istoric_chat("user", copie_utilizator);
    adauga_istoric_chat("assistant", raspuns_local);
    esp_err_t rezultat = trimite_json_kv(cerere, "reply", raspuns_local);
    free(raspuns_local);
    return rezultat;
}
```

Dacă mesajul nu poate fi tratat local, funcția apelează *chat_client_apel*, care trimite cererea către OpenAI. În cazul în care acest apel eșuează sau nu întoarce un răspuns valid, funcția transmite clientului un mesaj de eroare explicativ. Dacă apelul reușește, răspunsul primit de la AI este adăugat în istoricul conversației, alături de mesajul utilizatorului.

```
esp_err_t cod_eroare = chat_client_apel(copie_utilizator, &raspuns_remote, &detalii_eroare);
cJSON_Delete(obiect_json);
if (cod_eroare != ESP_OK || !raspuns_remote) {
    const char *mesaj_eroare = detalii_eroare ? detalii_eroare : "API OpenAI a eșuat.";
    esp_err_t rezultat = trimite_json_eroare(cerere, "500 Internal Server Error", mesaj_eroare);
    free(detalii_eroare);
    return rezultat;
}

adauga_istoric_chat("user", copie_utilizator);
adauga_istoric_chat("assistant", raspuns_remote);
```

La final, funcția construiește un obiect JSON de forma { "reply": "..." }, îl serializează și îl trimite browserului ca răspuns HTTP. Pe tot parcursul execuției, *gestioneaza_chat_post* eliberează memoria alocată temporar și șterge obiectele JSON intermediare, pentru a evita scurgerile de memorie, aspect esențial într-un sistem cu resurse limitate.

```
httpd_resp_set_type(cerere, "application/json");
httpd_resp_set_hdr(cerere, "Cache-Control", "no-store");
esp_err_t rezultat_trimitere = httpd_resp_send(cerere, text_raspuns, HTTPD_RESP_USE_STRLEN);
free(text_raspuns);
return rezultat_trimitere;
```

3.3.2. CHAT

1. **Structura *chat_entry_t***

```
struct chat_entry_t {
    char rol[10];
    char continut[CHAT_MESSAGE_MAX_LEN];
};
```

Structura *chat_entry_t* definește forma unei singure intrări din istoricul conversației dintre utilizator și asistent. Fiecare intrare conține două câmpuri esențiale: rol (care indică rolul mesajului în dialog) și conținut (care stochează efectiv textul mesajului). Dimensiunea câmpului rol este limitată la un număr mic de caractere, deoarece valorile posibile sunt cunoscute și scurte, iar câmpul conținut are o lungime maximă fixă, stabilită prin constanta *CHAT_MESSAGE_MAX_LEN*.

2. Constanta *ETICHETA_CHAT & PROMPT_SISTEM*

Constanta *ETICHETA_CHAT* reprezintă eticheta de log asociată modulului *chat_client* pentru a identifica rapid sursa mesajelor în jurnalul de depanare (log). Utilizarea unei etichete dedicate este recomandată, deoarece permite filtrarea și analizarea logurilor atunci când aplicația devine mai complexă.

```
static const char *ETICHETA_CHAT = "chat_client";
static const char *PROMPT_SISTEM = "Ești asistentul Elianei care rulează pe un mini-arduino. "
                                   "Răspunde succint, în limba română, într-un stil prietenos și de ajutor.";
```

Constanta *PROMPT_SISTEM* definește mesajul trimis către modelul OpenAI la fiecare cerere. Acest mesaj stabilește contextul global al conversației și comportamentul dorit al asistentului (prietenos), precizând identitatea acestuia, mediul de rulare (un microcontroler de tip „mini-arduino”) și stilul răspunsurilor (de ajutor).

3. Funcția *adauga_istoric_chat*

```
void adauga_istoric_chat(const char *rol, const char *continut)
{
    if (!rol || !continut) {
        return;
    }
    if (s_dimensiune_istoric >= CHAT_HISTORY_MAX_MESSAGES) {
        memmove(&s_istoric[0], &s_istoric[1], sizeof(chat_entry_t) * (CHAT_HISTORY_MAX_MESSAGES - 1));
        s_dimensiune_istoric = CHAT_HISTORY_MAX_MESSAGES - 1;
    }
    strlcpy(s_istoric[s_dimensiune_istoric].rol, rol, sizeof(s_istoric[0].rol));
    strlcpy(s_istoric[s_dimensiune_istoric].continut, continut, sizeof(s_istoric[0].continut));
    s_dimensiune_istoric++;
```

Funcția *adauga_istoric_chat* gestionează memoria de conversație locală. Rolul ei este să păstreze ultimele mesaje într-un istoric de dimensiune limitată, astfel încât la următorul apel către OpenAI asistentul să primească și contextul conversației.

```
    if (s_dimensiune_istoric >= CHAT_HISTORY_MAX_MESSAGES) {
        memmove(&s_istoric[0], &s_istoric[1], sizeof(chat_entry_t) * (CHAT_HISTORY_MAX_MESSAGES - 1));
        s_dimensiune_istoric = CHAT_HISTORY_MAX_MESSAGES - 1;
    }
```

Implementarea este gândită ca un istoric „circular” simplificat: dacă numărul maxim de mesaje este depășit, mesajul cel mai vechi este eliminat (prin mutarea elementelor în memorie, prin memmove),

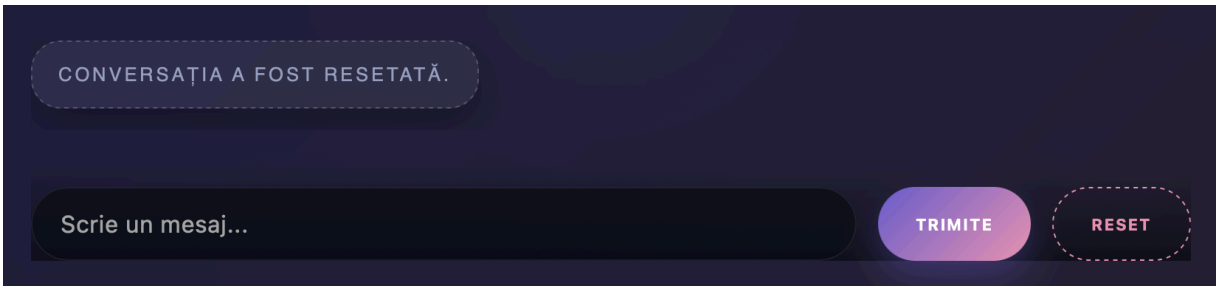
```
    strlcpy(s_istoric[s_dimensiune_istoric].rol, rol, sizeof(s_istoric[0].rol));
    strlcpy(s_istoric[s_dimensiune_istoric].continut, continut, sizeof(s_istoric[0].continut));
    s_dimensiune_istoric++;
```

iar mesajul nou este adăugat la final, păstrând astfel cele mai recente mesaje, care sunt și cele mai relevante pentru context. Prin această abordare, aplicația păstrează mereu cele mai recente interacțiuni, care sunt cele mai importante pentru continuitatea dialogului.

4. **Funcția *istoric_chat_goleste***

```
/** Șterge toate mesajele memorate. Folosit când utilizatorul apasă butonul Reset în UI. */
void istoric_chat_goleste(void)
{
    s_dimensiune_istoric = 0;
}
```

Funcția *istoric_chat_goleste* resetează istoricul conversației prin setarea contorului *s_dimensiune_istoric* la zero. Datele vechi rămân fizic în buffer, însă sunt considerate irelevante deoarece nu mai sunt incluse în cererile ulterioare. În interfața utilizatorului, funcția este utilă în special când utilizatorul apasă un buton de tip *„Reset”*, dorind o conversație *„de la zero”*, fără context anterior.



5. **Funcția *chat_client_apel***

Funcția *chat_client_apel* este componenta principală a modulului, practic, „motorul” comunicării cu OpenAI. Scopul ei este să ia un mesaj scris de utilizator, să îl trimită către API (Interfața de Programare a Aplicațiilor) și să îl întoarcă tot sub formă de text. Astfel, funcția acționează ca un intermediar între utilizator și inteligența artificială.

```
if (!mesaj_utilizator || strlen(mesaj_utilizator) == 0) {
    return ESP_ERR_INVALID_ARG;
}

if (strlen(CONFIG_APP_OPENAI_KEY) == 0) {
    ESP_LOGE(ETICHETA_CHAT, "Missing OpenAI API key. Set `App config -> OpenAI API Key`.");
    return ESP_ERR_INVALID_ARG;
}
```

Primul lucru pe care îl face este să verifice dacă mesajul utilizatorului este valid și dacă există cheia necesară pentru a accesa API-ul.

```
cJSON *radacina = cJSON_CreateObject();
cJSON *lista_mesaje = cJSON_CreateArray();
if (!radacina || !lista_mesaje) {
    cJSON_Delete(radacina);
    return ESP_ERR_NO_MEM;
}
```

```
for (size_t i = 0; i < s_dimensiune_istoric; ++i) {
    cJSON *mesaj_anterior = cJSON_CreateObject();
    cJSON_AddStringToObject(mesaj_anterior, "role", s_istoric[i].rol);
    cJSON_AddStringToObject(mesaj_anterior, "content", s_istoric[i].continut);
    cJSON_AddItemToArray(lista_mesaje, mesaj_anterior);
}
```

După verificări, funcția trimite cererea la OpenAI, fiind construită într-un format numit JSON. Acesta include un mesaj de sistem, mesajele din istoricul conversației și mesajul curent al utilizatorului. Acest lucru permite AI-ului să răspundă având context, nu doar pe baza ultimului mesaj.

```
cJSON *mesaj_sistem = cJSON_CreateObject();
cJSON_AddStringToObject(mesaj_sistem, "role", "system");
cJSON_AddStringToObject(mesaj_sistem, "content", PROMPT_SISTEM);
cJSON_AddItemToArray(lista_mesaje, mesaj_sistem);
```

```
cJSON_AddStringToObject(radacina, "model", CONFIG_APP_OPENAI_MODEL);
cJSON_AddItemToObject(radacina, "messages", lista_mesaje);
cJSON_AddNumberToObject(radacina, "temperature", 0.6);
cJSON_AddNumberToObject(radacina, "max_tokens", 120);
```

Construcția cererii JSON este făcută cu *cJSON.*, iar payload-ul include:

- un mesaj de tip *system (PROMPT_SISTEM)*, care definește stilul și constrângerile răspunsului;
- istoricul conversației stocat local (rol și conținut);
- mesajul curent al utilizatorului, cu rolul user;
- parametrii de generare (*model, temperature, max_tokens*), care controlează modelul folosit și „libertatea” răspunsului.



După ce JSON-ul este pregătit, acesta este transformat într-un text și trimis prin internet folosind un client HTTP configurat. În timpul trimiterii, un antet special cu cheia API spune serverului cine a făcut cererea și îi permite accesul. Dacă trimiterea reușește, serverul OpenAI răspunde înapoi tot în format JSON.

```
// Configurăm clientul HTTP pentru un POST simplu, folosind certificatul implicit.
esp_http_client_config_t config = {};
config.url = "https://api.openai.com/v1/chat/completions";
config.method = HTTP_METHOD_POST;
config.event_handler = gestioneaza_eveniment_http;
config.user_data = &raspuns_buffer;
config.disable_auto_redirect = true;
config.crt_bundle_attach = esp_crt_bundle_attach;
```

Transmiterea către OpenAI se face prin *esp_http_client* configurat pentru un POST către endpoint-ul *https://api.openai.com/v1/chat/completions*, folosind pachetul de certificate implicit *(esp_crt_bundle_attach)* pentru TLS.

```
char *antet_autentificare = NULL;
if (asprintf(&antet_autentificare, "Bearer %s", CONFIG_APP_OPENAI_KEY) < 0) {
    esp_http_client_cleanup(client_http);
    free(payload);
    return ESP_ERR_NO_MEM;
}

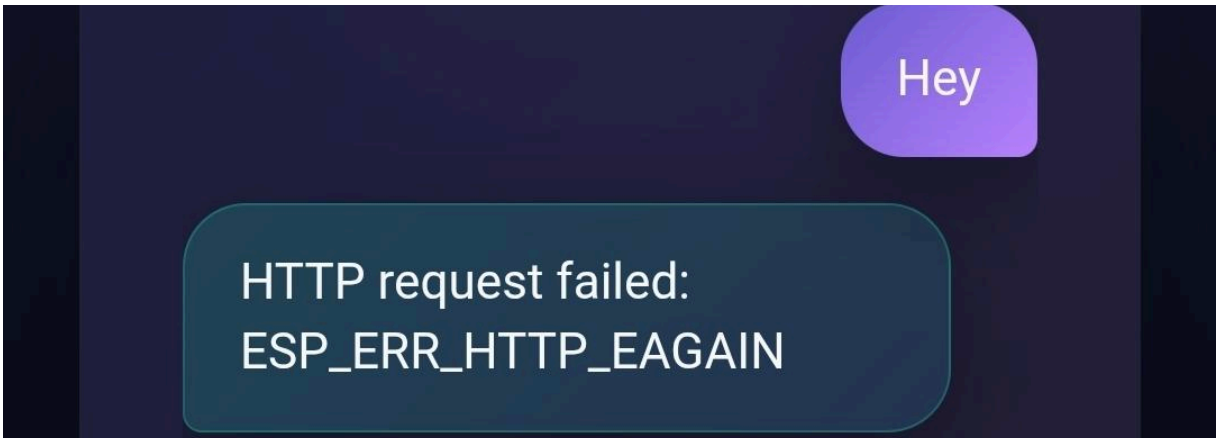
esp_http_client_set_header(client_http, "Content-Type", "application/json");
esp_http_client_set_header(client_http, "Authorization", antet_autentificare);
esp_http_client_set_post_field(client_http, payload, strlen(payload));
```

Se setează antetele necesare:
Content-Type:application/json și *Authorization: Bearer <API_KEY>*. Corpul cererii este payload-ul JSON generat anterior.

```
// Trimitem cererea; erorile aici sunt de regulă TLS/soket.
esp_err_t cod_eroare = esp_http_client_perform(client_http);
if (cod_eroare != ESP_OK) {
    ESP_LOGE(ETICHETA_CHAT, "HTTP request failed: %s", esp_err_to_name(cod_eroare));
    if (eroare_iesire) {
        asprintf(eroare_iesire, "HTTP request failed: %s", esp_err_to_name(cod_eroare));
    }
```

Funcția preia răspunsul și caută în el partea cea mai importantă, adică textul generat de AI. Dacă acel text există și este valid, el este trimis înapoi aplicației. Dacă serverul trimite coduri de eroare sau un răspuns incomplet, funcția oferă un mesaj de eroare pe care îl poate afișa pe interfața utilizatorului.

```
if (cod_status != 200) {
    if (raspuns_buffer.buffer && raspuns_buffer.lungime > 0) {
        ESP_LOGE(ETICHETA_CHAT,
                 "OpenAI returned HTTP %d: %.*s",
                 cod_status,
                 (int)raspuns_buffer.lungime,
                 raspuns_buffer.buffer);
    } else {
        ESP_LOGE(ETICHETA_CHAT, "OpenAI returned HTTP %d", cod_status);
    }
```




```
    if (eroare_iesire && raspuns_buffer.buffer) {
        cJSON *err_json = cJSON_ParseWithLength(raspuns_buffer.buffer, raspuns_buffer.lungime);
        if (err_json) {
            cJSON *err_obj = cJSON_GetObjectItem(err_json, "error");
            cJSON *msg = err_obj ? cJSON_GetObjectItem(err_obj, "message") : NULL;
            if (cJSON_IsString(msg) && msg->valuestring) {
                *eroare_iesire = strdup(msg->valuestring);
            }
            cJSON_Delete(err_json);
        }
        if (!*eroare_iesire) {
            size_t len = raspuns_buffer.lungime < 200 ? raspuns_buffer.lungime : 200;
            *eroare_iesire = copiaza_n_caractere(raspuns_buffer.buffer, len);
        }
    }
    cod_eroare = ESP_FAIL;
```

Dacă statusul este 200, se parsează corpul răspunsului ca JSON și se caută câmpul standard așteptat: *choices[0].message.content.* Dacă acesta există și este de tip șir, textul este duplicat și returnat prin *mesaj_raspuns.* În lipsa câmpului sau în cazul unui răspuns neinterpretabil, funcția semnalează un răspuns invalid și furnizează un mesaj explicativ în *eroare_iesire*.

```
cJSON *json = cJSON_ParseWithLength(raspuns_buffer.buffer, raspuns_buffer.lungime);
if (!json) {
    ESP_LOGE(ETICHETA_CHAT, "Failed to parse OpenAI response");
    if (eroare_iesire) {
        asprintf(eroare_iesire, "Nu am putut interpreta răspunsul OpenAI.");
    }
    free(raspuns_buffer.buffer);
    return ESP_ERR_INVALID_RESPONSE;
}
```

```
    if (eroare_iesire) {
        asprintf(eroare_iesire, "Răspuns invalid de la OpenAI.");
    }
}

cJSON_Delete(json);
free(raspuns_buffer.buffer);
return cod_eroare;
```

Pe tot parcursul, funcția are grijă să elibereze resursele temporare (payload, antet de autorizare, client HTTP, buffer de răspuns, obiecte cJSON), astfel încât execuțiile repetate să nu conducă la scurgeri de memorie.

3.3.3. LED

1. **Funcția *aplica_culoare_led***

```
static void aplica_culoare_led(uint8_t componenta_rosu, uint8_t componenta_verde, uint8_t componenta_albastru)
{
    uint8_t rosu_scalat = luminozitate(componenta_rosu);
    uint8_t verde_scalat = luminozitate(componenta_verde);
    uint8_t albastru_scalat = luminozitate(componenta_albastru);

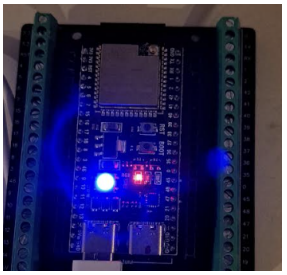
    if (s_controler_led) {
        esp_err_t err;
        if (rosu_scalat == 0 && verde_scalat == 0 && albastru_scalat == 0) {
            err = led_strip_clear(s_controler_led);
        } else {
            err = led_strip_set_pixel(s_controler_led, 0, rosu_scalat, verde_scalat, albastru_scalat);
            if (err == ESP_OK) {
                err = led_strip_refresh(s_controler_led);
            }
        }
        if (err != ESP_OK) {
            ESP_LOGW(ETICHETA_LED, "Failed to update LED strip: %s", esp_err_to_name(err));
        }
    } else if (s_led_gpio) {
        gpio_set_level(static_cast<gpio_num_t>(CONFIG_APP_LED_GPIO),
                       (rosu_scalat + verde_scalat + albastru_scalat) > 0);
    }
}
```

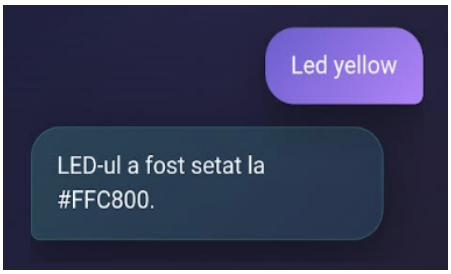
Funcția *aplica_culoare_led* are rolul de a aplica imediat o culoare asupra LED-ului de stare, indiferent de tipul de hardware disponibil. La început, funcția ajustează intensitatea fiecărei componente de culoare (roșu, verde și albastru) prin aplicarea unui factor global de luminozitate la 80% . Această scalare are scopul de a limita intensitatea maximă a LED-ului, prevenind atât un consum excesiv de energie, cât și un disconfort vizual.

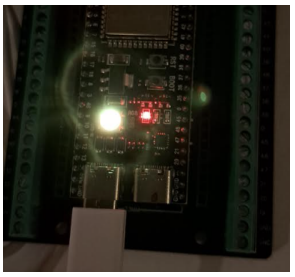
```
    uint8_t rosu_scalat = luminozitate(componenta_rosu);
    uint8_t verde_scalat = luminozitate(componenta_verde);
    uint8_t albastru_scalat = luminozitate(componenta_albastru);
```

Dacă sistemul a reușit inițializarea unui LED adresabil, funcția utilizează API-ul oferit de driverul *led_strip*. În cazul în care toate componentele de culoare sunt zero, LED-ul este stins complet prin apelul funcției *led_strip_clear*. În caz contrar, culoarea este aplicată pixelului corespunzător, iar modificarea este transmisă efectiv către hardware printr-un apel de reîmprospătare. Orice eroare apărută în proces este raportată în jurnalul de sistem.









```
    if (rosu_scalat == 0 && verde_scalat == 0 && albastru_scalat == 0) {
        err = led_strip_clear(s_controler_led);
    } else {
        err = led_strip_set_pixel(s_controler_led, 0, rosu_scalat, verde_scalat, albastru_scalat);
        if (err == ESP_OK) {
            err = led_strip_refresh(s_controler_led);
        }
```

În situația în care LED-ul adresabil nu este disponibil, dar a fost configurat un pin GPIO, funcția recurge la un comportament de tip *„fallback”*. În acest mod, LED-ul poate fi doar aprins sau stins, în funcție de existența unei componente de culoare nenulă, fără posibilitatea de a reda nuanțe diferite.

2. Funcția *parseaza_culoare_hex*

```
static bool parseaza_culoare_hex(const char *mesaj, uint8_t *componenta_rosu, uint8_t *componenta_verde, uint8_t *componen
{
    if (!mesaj) {
        return false;
    }
    const char *pozitie_hash = mesaj;
    while ((pozitie_hash = strchr(pozitie_hash, '#')) != NULL) {
        if (!pozitie_hash[1] || !pozitie_hash[2] || !pozitie_hash[3] || !pozitie_hash[4] || !pozitie_hash[5] || !pozitie_h
            pozitie_hash++;
            continue;
        }
```

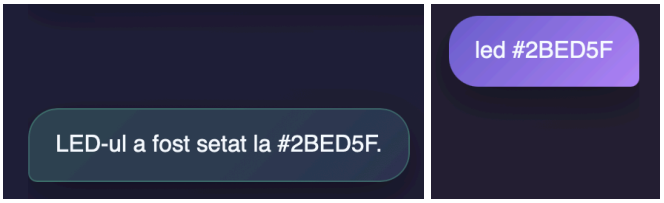
Funcția *parseaza_culoare_hex* are rolul de a identifica și interpreta coduri de culoare hexazecimala, de forma „#RRGGBB” dintr-un mesaj text. Aceasta permite astfel utilizatorului

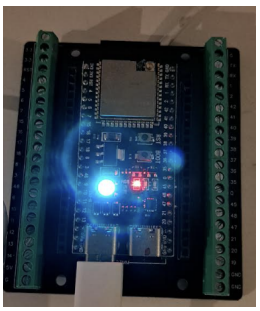
să specifice o culoare precisă folosind un format standard. Funcția parcurge șirul de caractere primit și caută apariții ale caracterului „#”.

Pentru fiecare poziție identificată, verifică dacă există suficiente caractere următoare pentru a forma un cod complet de culoare. Fiecare dintre cele șase caractere este validat pentru a confirma că aparține setului hexazecimal permis.

```
    int h3 = caracter_hex_la_int(pozitie_hash[3]);
    int h4 = caracter_hex_la_int(pozitie_hash[4]);
    int h5 = caracter_hex_la_int(pozitie_hash[5]);
    int h6 = caracter_hex_la_int(pozitie_hash[6]);
    if (h1 < 0 || h2 < 0 || h3 < 0 || h4 < 0 || h5 < 0 || h6 < 0) {
        pozitie_hash++;
        continue;
    }
    if (componenta_rosu) {
        *componenta_rosu = (uint8_t)((h1 << 4) | h2);
    }
    if (componenta_verde) {
        *componenta_verde = (uint8_t)((h3 << 4) | h4);
    }
    if (componenta_albastru) {
```

Dacă secvența este validă, funcția construiește valorile corespunzătoare componentelor roșu, verde și albastru, combinând câte două cifre hexazecimale pentru fiecare componentă. La prima potrivire corectă, valorile sunt returnate prin parametrii de ieșire, iar funcția semnalează succesul operației. În lipsa unei potriviri valide, funcția indică eșecul parsării.





3. Funcția *parseaza_culoare_nume*

Funcția *parseaza_culoare_nume* permite recunoașterea culorilor exprimate sub formă de cuvinte, în limba română sau engleză, oferind o interfață prietenoasă.



```
static const culoare_nume_t CULORI[] = {
    {"rosu", 255, 0, 0},
    {"roșu", 255, 0, 0},
    {"red", 255, 0, 0},
    {"verde", 0, 255, 0},
    {"green", 0, 255, 0},
```

```
    {"negru", 0, 0, 0},
    {"stins", 0, 0, 0},
    {"stinge", 0, 0, 0},
    {"opreste", 0, 0, 0},
    {"oprește", 0, 0, 0},
```

În implementare este definit un tabel static care asociază denumiri de culori cu valori RGB predefinite. Tabelul include atât culori uzuale (roșu, verde sau albastru), cât și comenzi semantice pentru stingerea LED-ului, cum ar fi *„off”* sau *„stins”*. Funcția verifică dacă mesajul primit conține oricare dintre aceste denumiri. La prima potrivire găsită, componentele de culoare asociate sunt copiate în variabilele de ieșire, iar funcția se încheie cu succes. Dacă niciuna dintre denumirile cunoscute nu este detectată în mesaj, funcția returnează eșec.

4. **Funcția *led_ctrl_proceseaza_comanda***

```
bool led_ctrl_proceseaza_comanda(const char *mesaj, char **raspuns_iesire)
{
    if (raspuns_iesire) {
        *raspuns_iesire = NULL;
    }
    if (!mesaj) {
        return false;
    }
```

Funcția *led_ctrl_proceseaza_comanda* reprezintă mecanismul principal de interpretare a comenzilor textuale adresate LED-ului. Ea analizează mesajele primite și decide dacă acestea conțin o comandă validă de control al iluminării.

```
    mesaj_mic[lungime] = '\0';

    bool contine_cuvant_cheie = strstr(mesaj_mic, "led") || strstr(mesaj_mic, "rgb") || strstr(mesaj_mic, "lumina
    if (!contine_cuvant_cheie) {
        return false;
    }
```

Funcția verifică dacă textul conține termeni relevanți pentru domeniul LED-urilor, precum *„led”*, *„rgb”* sau *„lumină”*. Această etapă previne interpretarea accidentală a unor mesaje care nu au legătură cu controlul LED-ului.

```
uint8_t componenta_rosu = 0, componenta_verde = 0, componenta_albastru = 0;
if (!parseaza_culoare_hex(mesaj_mic, &componenta_rosu, &componenta_verde, &componenta_albastru) &&
    !parseaza_culoare_nume(mesaj_mic, &componenta_rosu, &componenta_verde, &componenta_albastru)) {
    return false;
}
```

Dacă mesajul este considerat relevant, funcția încearcă să extragă o culoare, mai întâi sub formă de cod hexazecimal, iar apoi sub formă de denumire textuală. În cazul în care nu este identificată nicio culoare validă, mesajul este ignorat.

```
if (raspuns_iesire) {
    if (componenta_rosu || componenta_verde || componenta_albastru) {
        asprintf(raspuns_iesire, "LED-ul a fost setat la #%02X%02X%02X.",
    } else {
        *raspuns_iesire = strdup("LED-ul a fost stins.");
    }
}
```

Atunci când o culoare este recunoscută, aceasta este aplicată imediat LED-ului, iar acțiunea este consemnată în jurnalul de sistem. Opțional, funcția poate genera un mesaj de răspuns, care confirmă utilizatorului culoarea setată sau faptul că LED-ul a fost stins. Funcția returnează un rezultat pozitiv doar în cazul în care comanda a fost interpretată și executată cu succes.

3.3.4. UI (Interfata User-ului)



1. **Interfața grafică a aplicației (GUI)**

```html
<header>
  <h1>Eliana's Laboratory</h1>
  <p>Laborator interactiv pe microcontroller — răspunsuri scurte, în română.</p>
  <p id="status">Se conectează...</p>
</header>

<main>
  <div class="card">
    <ul id="log"></ul>
    <form id="chat-form">
      <input id="chat-input" type="text" placeholder="Scrie un mesaj..." autocomplete="off" required />
      <button type="submit">Trimite</button>
      <button type="button" id="reset-btn">Reset</button>
    </form>
  </div>
</main>
```

2. **Funcția *append (role, text)* – afișarea mesajelor în GUI**

```javascript
function append(role, text) {
  const li = document.createElement('li');
  li.textContent = text;
  li.className = role;
  log.appendChild(li);
  li.scrollIntoView({behavior: 'smooth', block: 'end'});
  return li;
}
```

Aceasta este funcția de bază pentru UI. Ea adaugă un mesaj nou în listă și îi aplică o clasă (me, bot, info) ca să fie stilizat corect. În plus, face scroll automat la ultimul mesaj, ca utilizatorul să vadă mereu conversația curentă.

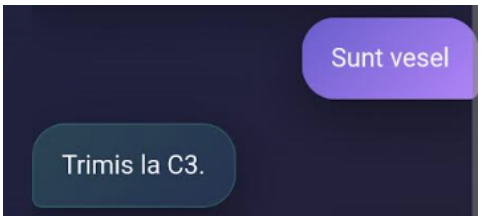
3.3.5. C3 OLED

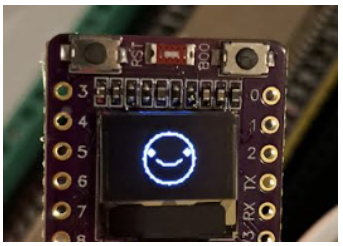
1. **Functia *display_show_icon***

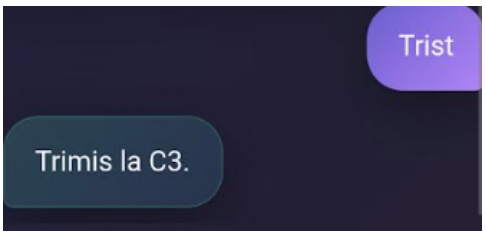
```
static const uint16_t icon_circle[16] = {
    0b0000011111100000,
    0b0001111111111000,
    0b0011111111111100,
    0b0111111111111110,
    0b0111111111111110,
    0b1111111111111111,
    0b1111111111111111,
    0b1111111111111111,
    0b1111111111111111,
    0b1111111111111111,
    0b0111111111111110,
    0b0111111111111110,
    0b0011111111111100,
    0b0001111111111000,
    0b0000011111100000,
};
```

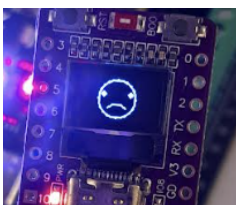
Există mai multe iconuri de forma 16×16, în care fiecare dintre cei 16 biți reprezintă un rând de pixeli (1=aprins). Acestea sunt folosite de funcția *display_show_icon()*care le transformă în formatul cerut și le scrie pe ecran și pot diferi drept:

- Emoticoane: *icon_smile, icon_sad;*
- Forme geometrice: *icon_square, icon_circle, icon_triangle,etc;*









```
} else if (strncasecmp(payload, "vesel", 5) == 0 ||
           strncasecmp(payload, "zambeste", 8) == 0 ||
           strncasecmp(payload, "smile", 5) == 0 ||
           strncasecmp(payload, "trist", 5) == 0 ||
           strncasecmp(payload, "sad", 3) == 0 ||
           strncasecmp(payload, "curata", 6) == 0 ||
           strncasecmp(payload, "patrat", 6) == 0 ||
           strncasecmp(payload, "triunghi", 8) == 0 ||
           strncasecmp(payload, "dreptunghi", 10) == 0 ||
```

2. **Functia *c3_client_proceseaza_comanda***

```
bool c3_client_proceseaza_comanda(const char *mesaj, char **raspuns_iesire)
{
    if (!mesaj) {
        return false;
    }

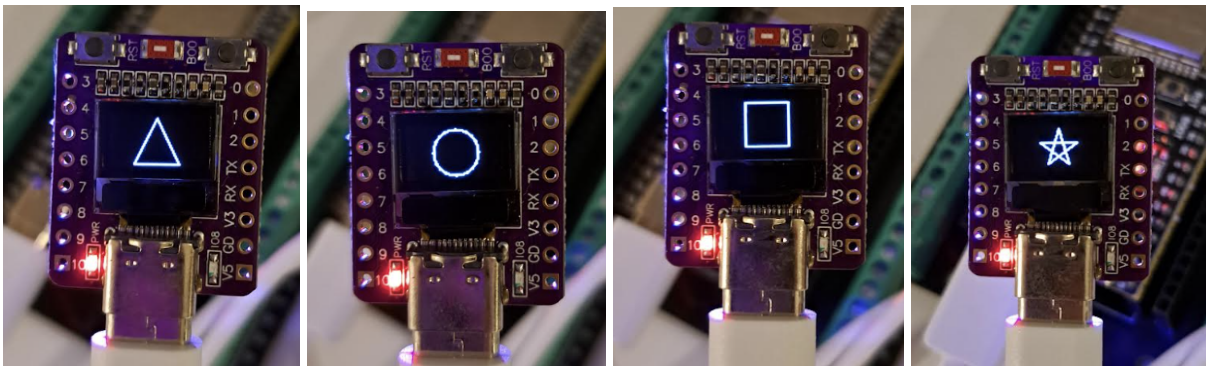
    // Acceptă: prefix "c3 <cmd>" sau direct cuvinte-cheie: vesel/trist/curata/clean + forme patrat/triunghi/dreptunghi/c
    const char prefix[] = "c3 ";
    const char *payload = mesaj;
    while (*payload == ' ') {
        payload++;
    }
    bool trigger = false;
    if (strncasecmp(payload, prefix, sizeof(prefix) - 1) == 0) {
        payload += (sizeof(prefix) - 1);
        while (*payload == ' ') {
            payload++;
        }
        trigger = true;
    }
```

Funcția *c3_client_proceseaza_comanda* are rolul de a detecta și executa comenzi destinate plăcii C3 *(denumită „C3-OLED”)*, folosind conexiunea Bluetooth Low Energy (BLE). Ea este gândită ca un „filtru local” similar cu modulul pentru LED, înainte ca mesajul să fie trimis către OpenAI, să verifice dacă textul introdus de utilizator este, de fapt, o comandă pentru dispozitivul C3 și, dacă da, o transmite direct prin GATT (Generic ATTribute Profile - folosit în comunicarea prin BLE pentru a schimba și accesa date între două dispozitive).

Funcția validează intrarea dacă mesajul este NULL și astfel nu are ce procesa și întoarce false. Apoi, normalizează modul în care identifică o comandă C3. Există două moduri prin care utilizatorul poate declanșa acest tip de comandă:

- cel explicit, printr-un prefix de forma *„c3”*, în care funcția ignoră spațiile de la început, și verifică dacă mesajul începe cu acest prefix (fără diferență între majuscule și minuscule, prin strncasecmp) și, dacă da, consideră că urmează o comandă pentru C3; prefixul este după eliminat și restul textului devine *„payload”*-ul efectiv
- cel implicit, fără prefix, în care funcția încearcă să recunoască mesaje care încep cu anumite cuvinte cheie ce sugerează intenția de a controla C3; în listă intră comenzi precum *„desenează”*, dar și termeni care descriu emoții (*„vesel”, „trist”*) sau formele

geometrice (*„patrat”, „triunghi”, „dreptunghi”, „cerc”, „stea” etc.*), precum și comenzi de curățare (*„curata”, „clean”, „clear”*); există inclusiv un caz special pentru propoziții de tip *„sunt ...”*, pentru a accepta formulări naturale precum *„sunt trist” sau „sunt vesel”*



Dacă nu este detectat nici prefixul, nici vreun cuvânt-cheie relevant, funcția concluzionează că mesajul nu este pentru C3 și întoarce *„false”*, lăsând sistemul să trateze mesajul pe alte căi (LED sau OpenAI).

Dacă mesajul a fost identificat ca fiind pentru C3, funcția mai face o curățare: dacă după declanșare textul începe încă o dată cu *„desenează”,* îl elimină, astfel încât comanda finală să fie cât mai concisă și stabilă.

După această etapă de interpretare, funcția verifică situațiile de eroare „logice”. Dacă nu mai rămâne nimic în payload (adică mesajul s-a terminat), funcția returnează succes *(„true”)*, dar întoarce un răspuns pentru UI de tipul *„Comanda C3 lipsă”*, ca utilizatorul să înțeleagă de ce nu s-a executat nimic. Apoi verifică dacă dispozitivul C3 este conectat și pregătit: trebuie să existe o conexiune BLE activă *(s_conn_handle valid)*, un ajutor de caracteristică identificat *(s_cmd_val_handle != 0)* și starea *s_ready* să fie *„true”*.

Dacă oricare dintre acestea lipsește, funcția tot consideră că mesajul a fost destinat C3, dar raportează către UI *„C3 nu este conectat încă”*. Această decizie este utilă: evită trimiterea către OpenAI a unor comenzi care sunt, de fapt, pentru hardware.

Înainte de transmitere, funcția limitează dimensiunea comenzii la maximum 60 de caractere, pentru a evita payload-uri prea mari pe BLE și pentru a menține un protocol simplu, apoi construiește o versiune „normalizată” a comenzii: anumite expresii naturale sunt mapate către comenzi standardizate, ușor de interpretat pe C3. De exemplu:

- ★ *„sunt trist”, „sad”, „trist”* devin emoji:sad;
- ★ *„sunt vesel”, „smile”, „zambeste”* devin emoji:smile;
- ★ *„triunghi”, „cerc”, „patrat”, „dreptunghi”, „stea” devin shape:<nume>;*
- ★ *„curata/clean/clear”* devin șterse.

Această normalizare este importantă pentru că separă limbajul natural al utilizatorului de *„protocolul”* simplu pe care îl înțelege placă C3, asigurând consistență și reducând ambiguitatea.

În final, funcția trimite efectiv comanda către C3 printr-un write GATT, folosind *ble_gattc_write_flat*, scriind în caracteristică identificată anterior la discovery. Dacă scrierea eșuează, se loghează eroarea și, opțional, se întoarce un mesaj pentru UI *(„Nu am reușit să trimit către C3.”)*. Dacă reușește, funcția confirmă prin *„Trimis la C3.”*.

4. Concluzia

Ca urmare, acest proiect demonstrează modul în care un sistem embedded modern poate depăși rolul clasic de control hardware și poate deveni o platformă interactivă, capabilă să interpreteze comenzi în limbaj natural și să răspundă inteligent utilizatorului. Prin integrarea unui microcontroler de tip ESP32 cu servicii de inteligență artificială, proiectul evidențiază legătura dintre programare și tehnologiile software avansate.

Aplicația realizată îmbină mai multe concepte din domeniul sistemelor embedded: conectivitate Wi-Fi, comunicare WebSocket și HTTP, procesarea locală a comenzilor, utilizarea memoriei non-volatile, controlul perifericelor (LED și ecran), conectarea la un API de inteligență artificială și comunicarea BLE cu un dispozitiv extern. Toate aceste componente sunt coordonate într-o arhitectură clară, în care interfața web, logica de procesare și controlul hardware sunt bine separate, ceea ce face codul ușor de urmărit și extins.

5. Bibliografie

- ★ Espressif Systems – ESP-IDF Programming Guide
  - https://docs.espressif.com/projects/esp-idf/
- ★ OpenAI – documentatia OpenAI API
  - https://platform.openai.com/docs/
- ★ Espressif – documentatia esp_http_client
  - https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/protocols/esp_http_client.html
- ★ Random Nerd Tutorials – ESP32 BLE Tutorial
  - https://randomnerdtutorials.com/esp32-bluetooth-low-energy-ble/
- ★ Espressif Systems – ESP32-C3 Datasheet
  - https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf
- ★ Espressif GitHub – ESP-IDF official repository
  - https://github.com/espressif/esp-idf
- ★ Mozilla Developer Network – WebSocket API Documentation
  - https://developer.mozilla.org/en-US/docs/Web/API/WebSocketa
- ★ Espressif – WiFi example ESP32
  - https://github.com/espressif/esp-idf/tree/master/examples/wifi
- ★ Wikipedia – Internet of Things
  - https://en.wikipedia.org/wiki/Internet_of_things

★ Espressif – documentatia ESP32 GPIO
    ○ https://docs.espressif.com/projects/arduino-esp32/en/latest/api/gpio.html
★ Wikipedia - C++
    ○ https://en.wikipedia.org/wiki/C%2B%2B